

Борьба с задержками

Prediction, Rollback и Lag Compensation



Что сегодня вскроем

ЛЕКЦИЯ 6

Ресар: четыре всадника задержки

CSP изнутри — что прячет Practice 3

Server Reconciliation — лечим расхождение

Lag Compensation — серверный VAR-rewind

DEMO: три призрака одной цели

Топ-7 ошибок и лайфхаки

Итоги и мост к Лекции 8

Вспомним Practice 3

ЛЕКЦИЯ 6

«Зелёный куб двигался плавно. Жёлтый — как пьяный. Почему?»

— любой студент после Practice 3, 200мс пинга

[Сцена Practice 3: два куба на одной траектории]
Жёлтый куб: ступенчатая ломаная траектория
Зелёный куб: гладкая дуга с интерполяцией

Сегодня мы открываем чёрный ящик.

RTT — это не «пинг». Это четыре отрезка времени.

ЛЕКЦИЯ 6

Client input $\xrightarrow{+[\Delta_1]}$ Server receive $\xrightarrow{+[\Delta_2]}$ Server send $\xrightarrow{+[\Delta_3]}$ Client render

t_0 t_1 t_2 t_3

$RTT = \Delta_1 + \Delta_2 + \Delta_3 \approx 2 \times \text{one-way}$

One-way $\approx RTT / 2$ (для prediction)

Interp delay = +1-2 tick (сглаживание на клиенте)

«Каждый игрок живёт в прошлом сервера ровно на one-way»

В Лекции 1 говорили про tickrate и framerate. Сегодня впервые смотрим внутрь одного тика.

Четыре всадника возвращаются. С именами.

ЛЕКЦИЯ 6

Latency

Задержка пути. Лечим: predict + lag comp.

Jitter

Нерегулярность задержки. Лечим: interp buffer.

Packet Loss

Потеря пакетов. Лечим: input redundancy + reliable RPC.

Bandwidth

Ширина канала. Лечим: компрессия (Лекция 2).

Сегодня нас интересуют первые два. Остальное — в багаже.

Эту проблему решили в 1996-м

ЛЕКЦИЯ 6



John Carmack / QuakeWorld, 1996

QuakeWorld, 1996

Кармак писал .plan-файлы вживую
Изобрёл client-side prediction
Input redundancy + lag compensation

*«Все идеи современного нетка —
оттуда. Вы 30 лет опоздали
изобрести колесо»*

Quake → Half-Life → Counter-Strike → CS:GO → Valorant. Линия преемственности.

Client-Side Prediction — одной фразой

ЛЕКЦИЯ 6

«Не ждать сервер. Симулировать у себя. Сервер потом подтвердит.»

```
// На каждом тике клиента:  
Input input = ReadInput();  
ApplyLocally(input);           // двигаемся СРАЗУ  
SendToServer(input, tick);     // и говорим серверу
```

🎮 Реакция: 0ms
📦 Цена: лжём себе,
готовимся к коррекции
⚠️ Только СВОЙ ввод. Не
чужой.

Сервер получает ваш ввод. Что он делает дальше?

ЛЕКЦИЯ 6

```
Client sent:  [t=10] [t=11] [t=12] [t=13] [t=14]
               ↓   ↓   ↓   ↓   ↓
Server:       t=10 → t=11 → t=12 → t=13 → t=14
Simulation:   tick = 10
```

«Это очередь в Макдональдсе. Сервер — один кассир, он не может обработать 4 заказа сразу. Если ваш ввод пришёл поздно — он попадёт в очередь следующего тика. Если слишком рано — подождёт».

**«Sequence number =
ваше место в
очереди. Без него —
анархия»**

Помните Practice 3? Вот за что отвечает каждая строчка.

ЛЕКЦИЯ 6

[Replicate]

```
void Move(MoveData md, ReplicateState state
    = ReplicateState.Invalid,
    Channel channel = Channel.Unreliable)
{
    Vector3 delta = md.Direction * speed
        * (float)TimeManager.TickDelta;
    transform.position += delta;
}
```

[Replicate]

Вызывается одинаково на клиенте и сервере

MoveData md

Ваш сериализованный input из input buffer

ReplicateState

Первый прогон или reconcile-replay

«Вы это писали. Просто никто не сказал зачем».

А если предсказание ошиблось?

ЛЕКЦИЯ 6

Клиент думал:

t=10: я в (5, 0)
t=11: нажал W → (5, 1)
t=12: продолжаю → (5, 2)

Сервер сказал:

t=10: ты в (5, 0) ✓
t=11: тебя оттолкнули → (3, 0) ✗

Что делать с тиком 12, который клиент УЖЕ просимулировал?

↓ ответ на следующем слайде ↓

Server Reconciliation — Ctrl+Z для физики

ЛЕКЦИЯ 6

1. Сервер шлёт snapshot: «на тике 11 ты был в (3, 0)»
2. Клиент откатывает себя на (3, 0)
3. Клиент повторно проигрывает все сохранённые входы от $t=12$ до now
4. Получается новая правильная позиция
5. Разница $< \text{threshold}$ — плавно интерполируем. Если $>$ — телепорт (rubber-band)

Параметры

State history: ring buffer 60-120 ticks

Threshold: 0.1-0.5 unit

Smoothing: lerp за 2-3 тика

Photoshop Ctrl+Z хранит историю. Ваш клиент — то же самое для физики.

Сколько прошлого помнит ваш клиент?

ЛЕКЦИЯ 6

120

ticks истории в Rocket
League

60 Hz × 2 секунды = 120 ticks

При 1.5-секундной задержке reconcile ещё
работает

Дальше — телепорт

«Больше истории = больше памяти. Trade-off».

Если ваша игра помнит 120 тиков, а вы — 3 секунды назад где были, поздравляю: вы хуже своего клиента.

Lag Compensation

Серверный rewind. Новая территория.

«Я попал! — Нет, не попал.»

ЛЕКЦИЯ 6

Клиент видит: **Враг ЗДЕСЬ** ■ (t=200ms, старое)
Клиент стреляет: ↓
Сервер (t=300ms): **Враг ЗДЕСЬ** ■ (свежее, правее)

Чьей правде верить?

Верим серверу «сейчас» → клиент мажет в 100% случаев.
Неиграбельно.
Верим клиенту слепо → читы. Любой может слать «я попал во всех».

Третий путь: сервер мотает
время назад

Серверный rewind: алгоритм

ЛЕКЦИЯ 6

1. Сервер хранит историю позиций ВСЕХ игроков (~1000ms)
2. Стрелок шлёт Shoot(direction, clientTick)
3. Сервер вычисляет $\text{rewindTime} = \text{RTT}/2 + \text{InterpDelay}$
4. Откатывает мир: все игроки → их позиции в $\text{serverTime} - \text{rewindTime}$
5. Делает Raycast в этом «прошлом мире»
6. Если попал — наносит урон жертве в её **НЫНЕШНЕЙ** позиции
7. Возвращает мир обратно

Trade-off

Жертва уже ушла за угол, но получает урон. «Killed behind cover».

Cap на rewind: обычно 200ms. Иначе — чит-вектор.

У lag compensation есть третий участник



Стрелок

«Стреляю — попадаю. Игра отзывчивая. Счастлив.»



Сервер

«Решаю однозначно.
Контроль читов сохранён.
Доволен.»



Жертва

«Я УЖЕ был за углом. Меня
всё равно убили. Зол.»

«Каждое решение в нетке — это выбор, кому будет больно»

DEMO

Три призрака одной цели



Что мы только что увидели

ЛЕКЦИЯ 6

 **Красный** — правда сервера. С пингом — дёргается у нас в окне.

 **Жёлтый** — клиентская интерполяция. Плавный, но отстаёт.

 **Зелёный** — predicted клиент. Совпадает с красным даже на 500мс.

«Глаза не врут. Prediction действительно бесплатный»

Топ-7 способов сломать ваш мультиплеер

ЛЕКЦИЯ 6

✗ 1. Предсказывать ЧУЖОЙ ввод — Сразу нет. Вы не знаете, что нажмёт враг. Это не предсказание, это галлюцинация.

✗ 2. Забыть error threshold — Любая микроразница → дёрганный rubber-band. Игроки скажут «у вас плохой нетку».

✗ 3. Хитбокс из mesh collider — Lag comp откатывает мир — на каждом выстреле ресемплинг сложного меша. Капсула — наш друг.

✗ 4. Lag comp без cap'a — Откат на 2 секунды — это уже не компенсация, это машина времени. Чит-вектор.

...и ещё три

ЛЕКЦИЯ 6

✗ 5. NetworkVariable для позиции в шутере — Это не для движения. Это для здоровья и счёта. Для движения — prediction + NetworkTransform.

✗ 6. Tickrate 20Hz + ожидание Valorant-feel — Никаких чудес. Valorant — 128Hz. Если у вас 20 — это казуальная игра, не шутер. Признайте.

✗ 7. Игнор packet loss — Один потерянный пакет = один потерянный ввод = реконсайд на полсекунды позже. Шлите последние 3 input в каждом пакете.

Лайфхак #7 (input redundancy) — копеечная стоимость, спасает 95% jitter-проблем

Три инструмента нетк-инженера

ЛЕКЦИЯ 6



Hit markers + server confirm

Клиент рисует крестик попадания МГНОВЕННО (по локальному prediction). Сервер позже либо подтверждает урон, либо тихо игнорирует. Игроку всегда отзывчиво, читерам — ничего.



Двойной tickrate

Физика на 60Hz (плавность), сеть на 30Hz (трафик). NetworkTransform умеет это из коробки.



Input redundancy

В каждом пакете шлешь последние N input'ов. Сервер дедуплицирует по sequence. UDP-потери становятся невидимыми.

Эти три приёма — в любой production-игре от 2010-х. Если нет — это видно.

Три инструмента, одна задача

**Predict + Reconcile + Lag Comp = ощущение
игры без пинга**

Predict — лечит ВАШ ввод. Делает локально.

Reconcile — лечит расхождение. Откатывает и проигрывает заново.

Lag Comp — лечит ИХ ввод. Сервер мотает мир назад.

Домашка

Расширить демо. Добавить 4-го призрака — LagCompensatedShooter. Он стреляет в мишень и видит hit-marker мгновенно. Сервер подтверждает с задержкой. Покажите эффект «убит за углом».

Следующая лекция (L8): Matchmaking, лобби, релиз. Серверы у нас уже есть (L7), осталось их заселить.